

第28回 能力を追加する:インタフェース

今回のゴール

- `interface` で「回復できる」という能力の約束を作り、`implements` でクラスに持たせられる。
- 継承の親子関係とは別に、違う種類のクラス(僧侶と敵のスライム)に同じ能力を持たせられる。
- 既存のバトルに、手順を追って機能を追加できる(仕様 → 変更箇所 → 実装 → 実行 → 確認)。

前回とのつながり

第27回で、バトルの基準版ができた。今回は「回復魔法」を追加する。回復できるのは、味方の僧侶だけではない。敵の中にも、仲間を回復するスライムを作りたい。

- `Character` に回復魔法を書くと、魔法使いや戦士まで回復できてしまう。
- 僧侶は `Character` の子、スライムは `Enemy` の子で、親が違う。Java では、1 つのクラスが `extends` できる親は 1 つだけ。

そこで、親子関係とは別に「回復できる」という能力だけを表す **インタフェース** を使う。

実習時のフォルダ構成

```
ワークスペース
├─ java28_sample
│   └─ src
│       └─ battle
│           ├─ (第27回の解答例と同じ 7 ファイル。Priest.java と Main.java を変更)
│           └─ Healer.java      ← 新しいインタフェース
└─ java28_exercise
    └─ src
        └─ battle
            ├─ (sample と同じ 8 ファイル)
            └─ HealSlime.java    ← 演習 2 - 1 で自分で作る
```

- 配布ファイル: `プログラム/第28回/sample/battle/`、`プログラム/第28回/exercise/battle/`
- 解答例: `プログラム/第28回/answer/battle/`
- インタフェースは、`battle` パッケージを右クリック → 「新規」→ 「インタフェース」で作る。

1. インタフェース:能力の約束

プログラム/第28回/sample/battle/Healer.java

```
package battle;

// インタフェース：「回復できる」という能力の約束
// implements したクラスは、cure メソッドを必ず書く
public interface Healer {
    void cure(Character target);
}
```

- `interface` には、メソッドの **名前・引数・戻り値の型** だけを書く(中身は書かない)。第23回の抽象メソッドと同じ形。
- `new Healer()` のように、インタフェースそのもののインスタンスは作れない。

2. implements:約束を守るクラス

プログラム/第28回/sample/battle/Priest.java

```
package battle;

// 僧侶：Character を受けつぎ、Healer（回復できる）の約束も守る
public class Priest extends Character implements Healer {
    public Priest(String name, int hp, int power) {
        super(name, hp, power);
    }

    @Override
    public void attack(Character target, int turn) {
        System.out.println(getName() + "は杖でたたいた！ " + target.getName() + "に" + getPower() + "のダメージ");
        target.damage(getPower());
    }

    // Healer の約束：回復の祈り（12 回復）
    @Override
    public void cure(Character target) {
        target.heal(12);
        System.out.println(getName() + "は回復の祈りをささげた！ " + target.getName() + "のHPが" + target.getHp() + "になった");
    }
}
```

- `extends Character implements Healer` は、「`Character` の子であり、`Healer` の約束も守る」という意味。
- `implements` したら、インタフェースのメソッド(`cure`)を **必ず** `public` で書く。インタフェースのメソッドは、自動的に `public` 扱いになっているため。

- インタフェースは、1 つのクラスにいくつでも `implements` できる(`implements A, B` のように `,` で区切る)。 `extends` は 1 つだけ。

	抽象クラス(<code>Character</code>)	インタフェース(<code>Healer</code>)
表すもの	「何の一種か」(親子関係)	「何ができるか」(能力)
書けるもの	フィールド、コンストラクタ、ふつうのメソッド、抽象メソッド	メソッドの約束
いくつ持てるか	<code>extends</code> は 1 つだけ	<code>implements</code> はいくつでも

3. インタフェース型で扱う

プログラム/第28回/sample/battle/Main.java (追加したところ)

```
String[] commands = { "攻撃", "薬草を使う", "にげる", "ためる", "回復魔法" };
```

```
case 5:
    // 回復できる (Healer の) 仲間が、いちばん HP の少ない仲間を回復する
    boolean healed = false;
    for (Character member : party) {
        if (member instanceof Healer) {
            Healer healer = (Healer) member;
            healer.cure(findWeakest(party));
            healed = true;
        }
    }
    if (!healed) {
        System.out.println("回復魔法を使える仲間がない!");
    }
    break;
```

```
// HP がいちばん少ないメンバーを返す
static Character findWeakest(ArrayList<Character> members) {
    Character weakest = members.get(0);
    for (Character member : members) {
        if (member.getHp() < weakest.getHp()) {
            weakest = member;
        }
    }
    return weakest;
}
```

- `member instanceof Healer` は、「`member` の中身は `Healer` の約束を守るクラスか」を調べる(第22回で読んだ `instanceof` と同じ)。
- `(Healer) member` で `Healer` 型として扱うと、`cure` を呼べる。
- `Main` は、回復しているのが僧侶かどうかを知らない。「回復できる者なら回復して」と命令しているだけ。後で回復できる職業を増やしても、この `case 5` は変えなくてよい。
- `findWeakest` は、「今までで一番少ない人」を覚えておき、もっと少ない人が見つかったら入れかえる、という最小値を探す形。

4. 実行する前に予想する

1 ターン目に 5 を入力するとどうなるか。僧侶は何ターン目に仲間になったか(第24回)を思い出す。

5. 実行結果(最初の部分)

```
コマンドを選ぶ > 5
回復魔法を使える仲間がない！
```

僧侶が仲間になった後で 5 を選ぶと、一番 HP の少ない仲間が 12 回復する(例)。

```
コマンドを選ぶ > 5
僧侶は回復の祈りをささげた！ 勇者のHPが21になった
```

6. エラーを体験する

1. `Priest.java` の `cure` の前の `public` を消して保存する。
2. 「より弱いアクセス権限を割り当てようとした」というエラーになる。インタフェースのメソッドは `public` なので、実装するときも `public` が必要。
3. 元に戻す。次に `Priest` の `cure` メソッド全体をコメントにして保存し、「`Healer` の `cure` を実装していない」エラーを確認する。確認したら元に戻す。

演習1 サンプルを実行する

`java28_sample` でサンプルを実行し、僧侶が仲間になる前と後で「回復魔法」を試す。

演習2 既存のバトルに機能を追加する

`java28_exercise` を作り、`battle` パッケージに `プログラム/第28回/exercise/battle/` の 8 ファイルを入れる。

機能を追加するときは、いきなり書き始めず、次の順番で進める。

1. 仕様:何ができるようになるかを 1 文で書く。
2. 変更箇所:どのファイルの、どのメソッドを変える(作る)か書き出す(第27回の「どこを変えると何が変わるか」の表を使う)。
3. 実装:1 か所ずつ書く。
4. 実行・確認:予想した結果になるか確かめる。

演習2-1 いやしスライムを作る

- 仕様:「なかまの敵を 10 回復できる(3 回まで)、いやしスライムが出てくる」
- 変更箇所:

- 新しいクラス `HealSlime` : `Enemy` を受けつぎ、`Healer` を `implements` する。コンストラクタは `Enemy` と同じ 4 つの引数。
- 回復できる残りの回数を表すフィールド(例: `private int cureCount = 3;`)を用意する。
- `cure` は、残りの回数が 1 以上なら `target.heal(10)` で回復して回数を 1 減らし、`いやしスライムはなかまを回復した！ ゴブリンのHPが22になった` の形で表示する。残りが 0 なら `いやしスライムは回復しようとしたが、力が残っていない！` と表示する。
- `Main` の敵のグループに、いやしスライム(HP 25、攻撃力 3、経験値 20)を追加する。
- この時点では、いやしスライムはふつうに攻撃するだけ。回復はまだしない。
- 回数に制限がないと、いやしスライムが自分を回復し続けて、バトルが終わらなくなることがある。

演習2-2 敵のターンで回復させる

- 仕様:「いやしスライムは、弱っている敵(HP が最大の半分より少ない)がいれば、攻撃の代わりに回復する」
- 変更箇所: `Main` の `enemyTurn`。弱っている敵は、配布ファイルにある `findWeakestEnemy(enemies)` で探せる。

ヒント:

```
Enemy weakest = findWeakestEnemy(enemies);
if (enemy instanceof Healer && weakest.getHp() < weakest.getMaxHp() / 2) {
    // Healer 型に変換して cure を呼ぶ
} else {
    // 今までの攻撃の処理
}
```

演習2-3 戦士をよろいで守る

- 仕様:「戦士は、受けるダメージを 2 へらせる(ただし最低 1 は受ける)」
- 変更箇所: `Warrior` クラスで、`Character` の `damage` をオーバーライドする。
- へらした後のダメージは、`super.damage(へらした値)` で `Character` のルール(HP を 0 未満にしない)に任せる。
- `戦士はよろいで受け止めた！ ダメージは5になった` と表示する。

ヒント: `Math.max(amount - 2, 1)` で、「2 へらす。ただし 1 より小さくしない」を書ける(第25回)。

完成条件(例:いやしスライムが回復している場面と、戦士がダメージをへらしている場面が表示される):

敵: 4体

- 1: スライム HP:20/20
- 2: ゴブリン HP:40/40
- 3: オーク HP:70/70
- 4: いやしスライム HP:25/25

ゴブリンの攻撃！ 戦士に7のダメージ

戦士はよろいで受け止めた！ ダメージは5になった

いやしスライムはなかまを回復した！ ゴブリンのHPが22になった

(敵の行動は乱数で決まるので、表示される順番や相手は毎回変わる)

発展(任意)

- 魔法使いにも `Healer` を `implements` させて、回復量 5 の回復魔法を持たせる。`Main` の `case 5` を変えなくても、魔法使いが回復に参加することを確認する。
- 新しいインタフェース `Guard` (`void guard();`)を作り、戦士に「ぼうぎょ」の能力を持たせる。

確認問題

問1 コンパイルできるか。

```
Healer h = new Healer();
```

問2 次のクラスはコンパイルできるか。

```
public class Fairy implements Healer {  
    void cure(Character target) {  
        target.heal(5);  
    }  
}
```

問3 次のコードはコンパイルできるか(クラスはサンプルと同じ)。

```
Healer h = new Priest("僧侶", 25, 4);  
h.attack(new Warrior("戦士", 40, 10), 1);
```

■ 答え

- 問1:できない(インタフェースはインスタンスを作れない)
- 問2:できない(`cure` に `public` がない。インタフェースのメソッドを実装するときは `public` が必要)
- 問3:できない(変数の型は `Healer`。`Healer` には `cure` しかないので、`attack` は呼べない。第22回と同じ考え方)

振り返り

- 回復の能力を `Character` ではなくインタフェースで作った理由を説明できるか。
- 抽象クラスとインタフェースの違いを、表を見ずに 1 つ以上言えるか。
- 「仕様 → 変更箇所 → 実装 → 実行・確認」の順に進められたか。

次回へのつながり

次回は、追加する機能を自分で考える。今回の「仕様 → 変更箇所 → 実装 → 実行・確認」の手順を、自分の考えた機能で使う。